



UNIVERSIDAD DE CASTILLA-LA MANCHA

PROYECTO DE PRÁCTICAS

-

ClipSmithery

Desarrollado por:

Adrián Alameda Alcaide

Álvaro García Martínez

ÍNDICE

1.1 Objetivo del proyecto.....	2
1.2 Alcance y funcionalidades implementadas	2
1.3 Arquitectura general	2
2. Requisitos de la aplicación.....	3
2.1 Requisitos funcionales	3
2.2 Requisitos no funcionales	4
2.3 Requisitos de entorno y ejecución.....	5
3. Decisiones y justificación de diseño e implementación.....	6
3.1 Arquitectura Modular y Separación Cliente/Servidor	6
3.2 Implementación del Módulo IA (Vite/React)	6
3.3 Diseño de Interfaz y Usabilidad	6
3.4 Robustez, Dependencias y Ejecución Simplificada	6
4. Problemas encontrados y su resolución	6
4.1 Dependencia de APIs externas de Inteligencia Artificial (Actualizado)	7
4.2 Problemas con la creación de ejecutables multiplataforma.....	7
4.3 Gestión de dependencias y entorno Python	8
4.4 Procesamiento multimedia en tiempo real.....	8
4.5 Organización del código y crecimiento del proyecto	8
4.6 Compatibilidad y facilidad de evaluación	9
5. Conclusiones.....	9
6. Manual de Usuario.....	10
6.1 Requisitos de Ejecución y Arranque	10
6.2 Panel Principal y Navegación	10
6.3 Uso de la Aplicación	11
6.4 Consideraciones Finales para el Usuario	12
7. Bibliografía	12

1. Descripción general

1.1 Objetivo del proyecto

ClipSmithery es una web/app orientada a conseguir que el usuario pueda hacer lo que quiera con su video, incorporando herramientas multimedia útiles para creadores y usuarios: procesado de vídeo en el navegador (compresión, cambio de resolución, generación de miniaturas) y módulos (vista previa tipo catálogo, recomendador por cuestionario, etc.), todo con una interfaz cuidada y coherente.

El objetivo principal es **demostrar dominio de APIs multimedia del navegador** (vídeo, canvas, MediaRecorder), buenas prácticas de UI/UX y una arquitectura limpia que permita añadir nuevas funcionalidades de forma incremental, manteniendo el proyecto “demo-friendly”.

1.2 Alcance y funcionalidades implementadas

El proyecto se organiza como un **panel principal** (menú) desde el que se accede a diferentes módulos independientes:

- **Compresión de vídeo (cliente):** reducción del tamaño manteniendo audio y calidad razonable, con barra de progreso y comparativa antes/después.
- **Cambio de resolución (cliente):** reescalado del vídeo a resoluciones típicas (1080p/720p/480p...), sin obligar a comprimirlo.
- **Generación de miniaturas (cliente):** extracción de frames del vídeo en distintos instantes para crear miniaturas tipo “catálogo”.
- **Vista previa (cliente):** “preview” automático al seleccionar/hover, simulando la experiencia de exploración.
- **Recomendador de Películas (CineMágico AI)** Test para recomendar contenido y ChatBot integrado. Módulo Anidado (Vite/React/TypeScript) compilado a estáticos.

1.3 Arquitectura general

Frontend Principal: HTML + CSS (estilo “premium/glass”) y JavaScript modular por funcionalidad.

Módulo de IA (CineMágico AI): El módulo del Recomendador se implementó como una aplicación anidada usando Vite, React y TypeScript. Se compila a archivos estáticos (carpeta dist/) para ser servido por el servidor principal.

Backend (Servidor Estático): Se utiliza un servidor ligero con Python (servidor HTTP simple lanzado por run_app.bat) para servir el directorio raíz y permitir el acceso a la aplicación compilada en recomendador/dist/.

1.4 Público objetivo / caso de uso

- **Usuario final / evaluador:** alguien que quiere probar rápidamente herramientas multimedia desde el navegador.
- **Caso típico:** “Subo un vídeo → comprimo o cambio resolución → genero miniaturas → pruebo preview y módulos”.

2. Requisitos de la aplicación

En este apartado se describen los **requisitos funcionales y no funcionales**, así como el **entorno necesario** para ejecutar la aplicación correctamente.

2.1 Requisitos funcionales

La aplicación debe permitir al usuario realizar las siguientes acciones:

1. Acceder a un panel principal

- Mostrar un menú inicial con todas las funcionalidades disponibles.
- Permitir navegar entre módulos de forma independiente (una funcionalidad por página).

2. Subir vídeos desde el dispositivo

- Aceptar formatos de vídeo comunes soportados por el navegador (MP4, WebM, etc.).
- Previsualizar el vídeo original antes de aplicar cualquier procesamiento.

3. Comprimir vídeos

- Reducir el tamaño del archivo manteniendo audio y una calidad visual aceptable.
- Mostrar el progreso del proceso mediante una barra de carga.
- Permitir comparar el vídeo original y el vídeo comprimido.
- Descargar el resultado final.

4. Cambiar la resolución del vídeo

- Reescalar el vídeo a distintas resoluciones estándar (1080p, 720p, 480p...).

- Mantener la duración y el audio del vídeo original.
- Mostrar el resultado en un reproductor independiente.
- Permitir la descarga del vídeo reescalado.

5. Generar miniaturas automáticamente

- Extraer imágenes del vídeo en distintos instantes.
- Mostrar las miniaturas en un grid estilo catálogo.
- Permitir descargar las miniaturas generadas.

6. Vista previa automática

- Mostrar una previsualización del contenido antes de acceder al módulo.
- Simular la experiencia de exploración de una plataforma de streaming.

7. Funcionalidades basadas en IA (modo opcional / experimental)

- Integrar módulos como recomendador de contenido o resumen inteligente.
- Diseñar estas funcionalidades de forma que **no sean críticas** para el funcionamiento general de la aplicación, permitiendo su desactivación o simulación en caso de no disponer de conexión o API externa.

2.2 Requisitos no funcionales

1. Usabilidad

- Interfaz clara, visual y coherente, inspirada en plataformas de streaming reales.
- Navegación sencilla, sin necesidad de conocimientos técnicos.
- Uso de iconos, animaciones y feedback visual (hover, barras de progreso).

2. Portabilidad

- La aplicación debe poder ejecutarse en cualquier equipo con navegador moderno.
- Se proporciona un mecanismo de ejecución simplificado (doble clic) para facilitar la prueba por parte del evaluador.

3. Rendimiento

- Procesamiento de vídeo realizado mayoritariamente en el cliente para evitar sobrecarga del servidor.
- Uso de resoluciones y bitrates moderados para mantener tiempos de espera razonables.

4. Robustez

- La aplicación no debe fallar si una API externa no está disponible.
- Manejo de errores básicos (vídeo no válido, cancelación de procesos, etc.).

5. Mantenibilidad

- Código organizado por módulos (un archivo HTML/JS por funcionalidad).
- Estructura clara que permita añadir nuevas funcionalidades en el futuro.

2.3 Requisitos de entorno y ejecución

Requisitos mínimos del sistema

- **Sistema operativo:** Windows, Linux o macOS.
- **Navegador web moderno:** Google Chrome, Microsoft Edge o Firefox actualizado.
- **Conexión a Internet:** opcional (solo necesaria para funcionalidades IA experimentales).

Requisitos de software (modo completo)

- **Python 3.x** (solo si se utiliza el backend).
- **Librerías Python:**
 - Flask
 - Flask-CORS
 - Pillow
 - Otras librerías opcionales según funcionalidades IA
- **Navegador web** para la interfaz de usuario.

Ejecución simplificada

- Se incluye un script de ejecución que permite lanzar la aplicación con un doble clic, sin necesidad de ejecutar comandos manuales.
- El servidor backend se inicia automáticamente y abre la aplicación en el navegador.

3. Decisiones y justificación de diseño e implementación

3.1 Arquitectura Modular y Separación Cliente/Servidor

La aplicación se estructuró con una arquitectura modular: cada funcionalidad principal tiene su propia página HTML y JavaScript, lo que facilita el mantenimiento y la depuración. Se tomó la decisión fundamental de realizar el procesamiento multimedia (Compresión, Reescalado, Miniaturas) directamente en el cliente (navegador), utilizando APIs nativas como Canvas API y MediaRecorder. Esto reduce la carga del servidor.

3.2 Implementación del Módulo IA (Vite/React)

El módulo Recomendador de Películas (CineMágico AI) se implementó como una aplicación anidada utilizando Vite y React/TypeScript. Esta decisión permitió aprovechar las ventajas del ecosistema React para una interfaz de usuario avanzada (ChatBot, *hooks* de estado) y mantener el código de la IA separado y modular.

3.3 Diseño de Interfaz y Usabilidad

El diseño gráfico se inspira en plataformas de *streaming*, utilizando el estilo *glassmorphism* y una distribución en *grid*. Priorizando la usabilidad, haciendo la aplicación intuitiva y visualmente atractiva.

3.4 Robustez, Dependencias y Ejecución Simplificada

Se empleó un servidor ligero de Python (http.server lanzado por run_app.bat) para servir el proyecto en su conjunto. Se puso especial énfasis en la robustez, diseñando la aplicación para que el funcionamiento principal no falle si las APIs externas de IA no están disponibles o agotan su cuota. Se incluyó un script de ejecución (run_app.bat) para lanzar la aplicación con un doble clic, evitando la necesidad de comandos manuales y facilitando la evaluación.

4. Problemas encontrados y su resolución

Durante el desarrollo del proyecto encontramos diversos problemas de carácter técnico y de diseño, especialmente relacionados con el procesamiento multimedia, la ejecución multiplataforma y la integración de funcionalidades

avanzadas. A continuación, describimos los principales problemas detectados y las soluciones adoptadas.

4.1 Dependencia de APIs externas de Inteligencia Artificial

Problema: La funcionalidad de IA (especialmente el ChatBot y la Generación de Pósteres) consume modelos de alto uso (Gemini 3 Pro, Gemini 2.5 Image), agotando la cuota del nivel gratuito, lo que provoca errores constantes 429 RESOURCE_EXHAUSTED.

Resolución: En lugar de eliminar la funcionalidad:

1. **Justificación de Mantenimiento:** Decidimos mantener el ChatBot y la Generación de Pósteres como funcionalidades experimentales clave, bajo el supuesto de que la cuota se restablecerá.
2. **Mitigación de Dependencias:** El error de cuota se aisló al módulo del Recomendador, garantizando que el funcionamiento principal de la suite Clip Smithery (compresión, reescalado, etc.) sigue siendo robusto y 100% operativo sin depender de la IA.

4.2 Problemas con la creación de ejecutables multiplataforma

Problema:

Intentamos inicialmente generar un ejecutable .exe utilizando PyInstaller desde entornos Linux/WSL. Esto provocó:

- Ejecutables incompatibles con Windows.
- Errores de ejecución como “no se puede ejecutar esta aplicación en el equipo”.
- Problemas de conexión al servidor local.

Resolución:

Nos dimos cuenta de que PyInstaller genera ejecutables específicos del sistema operativo en el que se ejecuta.

Como solución:

- Recompilamos el ejecutable desde Windows nativo.
- Desactivamos opciones como debug y use_reloader en Flask para evitar ejecuciones múltiples.
- Finalmente decidimos emplear un método de ejecución más robusto y sencillo (script de arranque con doble clic), evitando dependencias innecesarias.

4.3 Gestión de dependencias y entorno Python

Problema:

Durante el desarrollo aparecieron errores por dependencias no instaladas (por ejemplo, PIL, flask-cors, dotenv), así como restricciones del sistema para la ejecución de scripts en PowerShell.

Resolución:

- Creamos un archivo requirements.txt con todas las dependencias necesarias.
- Documentamos el proceso de instalación de dependencias.
- Evitamos exigir al usuario final el uso de entornos virtuales o comandos manuales.
- Automatizamos el arranque del servidor y la apertura del navegador.

4.4 Procesamiento multimedia en tiempo real

Problema:

El procesamiento de vídeo en el navegador presenta limitaciones importantes:

- Diferencias de soporte entre navegadores.
- Consumo elevado de recursos.
- Dificultad para mantener audio y vídeo sincronizados durante la compresión o reescalado.

Resolución:

- Realizamos pruebas con distintos enfoques usando APIs nativas.
- Ajustamos parámetros como resolución, bitrate y framerate para garantizar estabilidad.
- Implementamos barras de progreso y feedback visual para mejorar la experiencia de usuario.
- Priorizamos la compatibilidad y fiabilidad frente a resultados excesivamente agresivos.

4.5 Organización del código y crecimiento del proyecto

Problema:

A medida que añadíamos funcionalidades, el código tendía a crecer rápidamente, dificultando su mantenimiento.

Resolución:

- Reorganizamos el proyecto en módulos independientes.
- Cada funcionalidad cuenta con su propio HTML y JavaScript.
- Evitamos la duplicación de código.
- Mejoramos la estructura general del proyecto para facilitar futuras ampliaciones.

4.6 Compatibilidad y facilidad de evaluación

Problema:

Uno de los retos principales era garantizar que el proyecto pudiera ejecutarse fácilmente en un entorno distinto al de desarrollo.

Resolución:

- Simplificamos al máximo el proceso de ejecución.
- Diseñamos el proyecto para funcionar correctamente sin configuraciones adicionales.
- Documentamos los pasos necesarios para su uso.

Este conjunto de problemas y soluciones nos permitió mejorar notablemente la calidad final del proyecto, reforzando su estabilidad y usabilidad.

5. Conclusiones

El desarrollo del proyecto **ClipSmithery** nos ha permitido aplicar de forma práctica los conceptos fundamentales de la asignatura de Multimedia, combinando procesamiento de vídeo, diseño de interfaces y arquitectura web moderna en una aplicación funcional y coherente.

A lo largo del proyecto hemos conseguido implementar un conjunto de funcionalidades multimedia reales, como la compresión de vídeo, el cambio de resolución, la generación de miniaturas y la simulación de una gran experiencia de usuario mediante vistas previas y navegación visual. Estas funcionalidades demuestran un dominio avanzado de las APIs multimedia del navegador y una correcta gestión del flujo de audio y vídeo.

Uno de los principales aprendizajes ha sido la importancia de **diseñar aplicaciones robustas y reproducibles**, evitando dependencias externas innecesarias. La experiencia con APIs de inteligencia artificial nos ha mostrado las limitaciones de los servicios externos en entornos académicos, lo que nos llevó a replantear el enfoque hacia soluciones locales o simuladas, priorizando la estabilidad.

Desde el punto de vista del diseño, hemos comprobado cómo una interfaz cuidada y una buena experiencia de usuario aportan un valor añadido significativo al proyecto, mejorando su percepción global sin comprometer el rendimiento ni la claridad funcional. La inspiración en plataformas de streaming reales ha permitido crear una aplicación visualmente atractiva y alineada con el concepto propuesto.

En conjunto, el proyecto no solo cumple los objetivos que nos planteamos inicialmente, sino que además deja una base sólida y extensible sobre la que se podrían añadir nuevas funcionalidades en el futuro, como análisis de contenido más avanzado o recomendaciones personalizadas. El trabajo que hemos realizado nos ha permitido demostrar que podemos realizar una integración de conocimientos técnicos, que tenemos capacidad de resolución de problemas y toma de decisiones fundamentadas.

6. Manual de Usuario

6.1 Requisitos de Ejecución y Arranque

Para ejecutar la aplicación se requiere tener instalado Python 3.x (para el servidor web local) y haber compilado previamente el módulo de IA con Node.js/npm (ejecutando `npm run build` en la carpeta recomendador/).

El proceso de inicio ha sido simplificado al máximo:

1. Asegúrese de estar en el directorio raíz del proyecto (ClipSmithery/).
2. Ejecute el script `run_app.bat`.
3. El script iniciará el servidor HTTP de Python en el puerto 5000.
4. Automáticamente, el navegador web predeterminado se abrirá en la URL: `http://127.0.0.1:5000/menu.html`, mostrando el menú principal de la aplicación.

6.2 Panel Principal y Navegación

El Panel Principal (`menu.html`) actúa como un *dashboard* central donde todas las funcionalidades están organizadas en tarjetas (`menu-card`). La navegación se realiza haciendo clic en la tarjeta deseada.

Las funcionalidades están agrupadas bajo títulos que facilitan la comprensión:

Grupo	Título	Contenido
Grupo 1	Herramientas Esenciales de Post-Producción	Contiene los módulos principales de manipulación directa de vídeo (Compresión, Cambio de Resolución, etc.).
Grupo 2	Servicios de Análisis Visual	Contiene las funcionalidades de extracción de datos y análisis de píxeles (Análisis de Paleta de Color, etc.).
Grupo 3	Servicios Asistidos por IA	Contiene el módulo anidado del Recomendador de Películas.

Función de Expansión: Las tarjetas que no caben inicialmente están ocultas. El usuario puede hacer clic en el botón "Mostrar X funcionalidades más ▼" para expandir el grupo y acceder a todas las herramientas disponibles.

6.3 Uso de la Aplicación

Módulos de Procesamiento Multimedia (Grupos 1 y 2)

El flujo de trabajo es consistente en todos los módulos (Compresión, Reescalado, Recorte, etc.):

- Selección del Archivo:** Utilice el botón para cargar el archivo de vídeo o audio desde el dispositivo.
- Configuración:** (Si aplica) Seleccione la opción deseada (ej. la resolución de salida, el filtro de color).
- Ejecución:** Pulse el botón de acción (ej. "Comprimir", "Aplicar Filtro").
- Descarga:** Una vez que el proceso finaliza, el vídeo procesado estará disponible para su descarga en el dispositivo.

Módulo Recomendador de Películas (CineMágico AI)

Este módulo es una aplicación de React anidada y tiene un flujo de trabajo específico:

- Acceso:** En el menú principal, haga clic en la tarjeta de Test de Películas (IA).

2. **Inicio del Test:** La aplicación se abre en la vista de introducción. Haga clic en "Iniciar Análisis de Perfil".
3. **Cuestionario:** Responda las preguntas del cuestionario. La aplicación utiliza estas respuestas para generar el *prompt* de la IA.
4. **Visualización de Resultados:** Tras el procesamiento (Loading View), se muestra la Cartelera Personalizada que incluye:
 - **Recomendaciones de Texto:** Listado de películas con una justificación (razón).
 - **Generación de Pósteres:** Por cada película, la aplicación intenta generar un póster visualmente único utilizando la API de Gemini Image.
5. **Uso del ChatBot:** El ChatBot, visible en el lateral, puede ser utilizado para preguntas de seguimiento o *feedback* sobre las recomendaciones. Tenga en cuenta que, al utilizar modelos avanzados de IA, esta función puede agotar la cuota de la API (error 429 RESOURCE_EXHAUSTED), siendo una función experimental.
6. **Reiniciar:** Utilice el botón "Reiniciar Análisis" para volver a la vista de introducción y realizar un nuevo test.

6.4 Consideraciones Finales para el Usuario

- **Rendimiento:** Se recomienda utilizar vídeos de tamaño moderado. Los procesos se ejecutan en el navegador, y el rendimiento depende directamente del hardware del dispositivo.
- **Conexión a Internet:** Los módulos de IA (Recomendador y ChatBot) requieren una conexión activa a Internet y una clave API válida.
- **Límites de Cuota (IA):** Las funciones de Generación de Pósteres y ChatBot pueden fallar temporalmente si se excede la cuota gratuita de la API de Gemini. El usuario deberá esperar a que la cuota se restablezca (generalmente en 24 horas).

7. Bibliografía

Para el desarrollo del proyecto hemos consultado diversas fuentes técnicas, tanto documentación oficial como recursos de apoyo relacionados con multimedia y desarrollo web.

- Mozilla Developer Network (MDN).
HTML Video, Canvas API, MediaRecorder API.
<https://developer.mozilla.org/>
- Flask Documentation.
Flask Web Framework.
<https://flask.palletsprojects.com/>
- Python Documentation.
Python 3 – Standard Library.
<https://docs.python.org/3/>
- Pillow Documentation.
Python Imaging Library (PIL Fork).
<https://pillow.readthedocs.io/>
- Google Fonts.
Tipografía Poppins.
<https://fonts.google.com/>